LazyCode: Compiling Interactive Intent into Batch Execution

A Query-Optimizer Architecture, Its Economics, and the Limits of Overnight Coding Agents

Gurunath Lunkupali Venugopal*

August 2026

Abstract

Coding agents call models the way a person would: one synchronous request at a time, at the full realtime price. But much of what agents do best — backlog burndown, migrations, test-coverage pushes — has nobody waiting, and every major provider already sells the product for that shape: a batch API, 24-hour window, half price. LazyCode compiles deferrable model calls onto that tier. A realtime model *plans*; the plan then runs overnight as barrier-synchronized *waves* of self-sufficient batch calls, and the barriers, memoization and three-window crash recovery are what make paying a provider unattended safe. The design transplants database query processing: operator algebra, rewrite rules, physical waves, a write-ahead log. We separate what is implemented (18,415 LOC, 413 default-suite tests) from what is merely designed or vocabulary. On cost, batch's ~50% discount on output tokens is unconditional; against a *prompt-cached* interactive baseline it wins on input only when T interactive turns compile to $R < T/5$ batch rounds. Where batch and cache discounts stack — provider-dependent, unverified here — width relaxes that condition, reaching $R < 2T$ only in the $N \to \infty$ limit. One small upstream change — deferred *model* requests — is what blocks durable-execution frameworks from treating model calls like deferred tools.

1 Introduction

Every mainstream coding agent — Claude Code, Codex, and their descendants — shares one interaction contract: a loop of synchronous model calls, each awaited by a harness, each priced at the realtime rate. The contract is right when a person is watching. It is an unexamined default when nobody is: the overnight backlog task, the 400-file mechanical migration, the test-coverage push that nobody wants to babysit. For that work, latency tolerance is measured in hours, and the providers already sell a product priced for it — OpenAI Batches and Anthropic Message Batches both offer a 24-hour completion window at 50% of the online price.

The catch is structural, not financial. A batch call cannot ask a follow-up question, cannot read the result of the previous call in the same submission, and may take hours to return. An agent loop is the opposite shape: deeply sequential, incrementally contextualized, interactively repaired. Moving agentic work to the batch tier therefore requires *compiling* it: transforming a deep interactive loop into a shallow, wide, self-sufficient plan before the first batch call is made.

LazyCode is that compiler, and this paper is its story — told with database-systems vocabulary because the mapping is the design, not decoration. A realtime model emits a *logical plan*: a typed DAG over an eight-operator algebra of engineering work. An *optimizer* rewrites it. A *physical planner* lowers it onto provider batch APIs as barrier-synchronized waves, grouped like batched row processing. An event-sourced

*Code: `github.com/rajagurunath/lazycode`. Companion serving-side paper: Tidal (`github.com/rajagurunath/tidal`).

log makes the whole execution crash-safe and exactly-once in effect, the way a write-ahead log makes a database recoverable. Data systems spent forty years moving from batch to streaming; agent systems, born streaming, have never asked what moving the other way would buy.

What it buys is governed by one inequality (Section 3): the honest baseline is not an uncached agent but a prompt-cached one paying 0.1× for repeated context, so batch's flat 0.5× wins on input tokens only when the plan collapses T interactive turns into $R < T/5$ batch rounds. Output tokens — uncacheable on either side — are an unconditional ~50% win. This single condition unifies the latency mechanism and the cost mechanism: the shallow-wide DAG is simultaneously what makes 24-hour latency tolerable *and* what makes the economics beat a well-engineered interactive baseline. We further show (Section 3.1) that where batch and prompt-cache discounts stack (provider-dependent; Section 3.1), the condition is width-dependent: a wave of $N \geq 3$ prefix-sharing calls needs only $R \cdot (0.05 + 0.95/N) < 0.1\,T$, which for wide waves relaxes toward $R < 2T$ — moving the design burden from *manufacturing depth-collapse* to *supplying width*, with consequences for generalization that Section 9 develops.

We make four contributions:

1. **An architecture** (Section 4): the query-processing transplant in full — operator algebra, rewrite rules with database analogues, content-addressed waves, and a three-window crash-safety protocol for paid third-party submissions. An implementation-truth account (Section 8) separates shipped machinery from declared vocabulary.

2. **An economics analysis** (Section 3): the $R < T/5$ derivation, its width-dependent refinement under discount stacking, and the confrontation with `service_tier: flex`, the zero-architecture route to the same discount that any batch-tier argument must now name as its baseline.

3. **The self-sufficiency discipline** (Section 5): front-loaded context harvest under byte budgets, contract-constrained outputs, and the *assumption ledger*, which converts a blocking clarification into a decision flagged for morning review.

4. **A generalization verdict** (Section 9): a four-part criterion for when the plan-online, execute-on-batch pattern pays, a transfer analysis into Pydantic AI + DBOS and NVIDIA's NeMo Agent Toolkit, and deferred model requests identified as the missing upstream seam. Recommendation: build the wave executor as a library; do not port the planner.

Throughout, we distinguish three epistemic layers explicitly: what is *implemented and tested* (18,415 LOC, 413 default-suite tests), what is *designed with plumbing in place*, and what is *vocabulary only*. Research prototypes usually blur these; a paper about honest economics should not.

2 Background: the two products and the honest baseline

Batch APIs. Both major providers accept a file (OpenAI) or list (Anthropic) of independent requests, promise completion within 24 hours, and charge half the synchronous price. Results arrive as a set kept for about a month. Three properties shape everything downstream: items are *independent* (no item can read another's output), *non-interactive* (no mid-flight clarification), and *opaquely scheduled* (providers publish no latency percentiles; most items complete in under an hour, but the tail is unbounded up to expiry). Neither provider supports per-item cancellation — a fact that reshaped our hedging design (Section 6).

The honest interactive baseline. A naive comparison prices the interactive agent at list rates and reports 60–85% savings. That baseline is a strawman: production agents use prompt caching, under which repeated context is billed at 0.1× base on cache reads. Our first design document made exactly this error; an adversarial

review caught it, and the corrected analysis (Section 3) is one of this paper's contributions. The strawman survives in most published comparisons of batch pricing.

The newer complication. Since 2026, the choice is no longer binary. OpenAI's `service_tier: flex` charges batch rates on ordinary synchronous calls — slower, capacity-contingent (429 on shortage, unbilled), but requiring *zero* architectural change. And Anthropic's batch worker now runs the *server-side* agentic loop (web search, code execution, MCP connectors); provider materials have additionally advertised deeper per-turn iteration limits and larger maximum outputs on the batch path than the synchronous one — vendor-reported figures we have not independently verified, and which move with each release. The batch tier is thus no longer "the same API, slower and cheaper": it is architecturally different in both directions, and Section 9 weighs both.

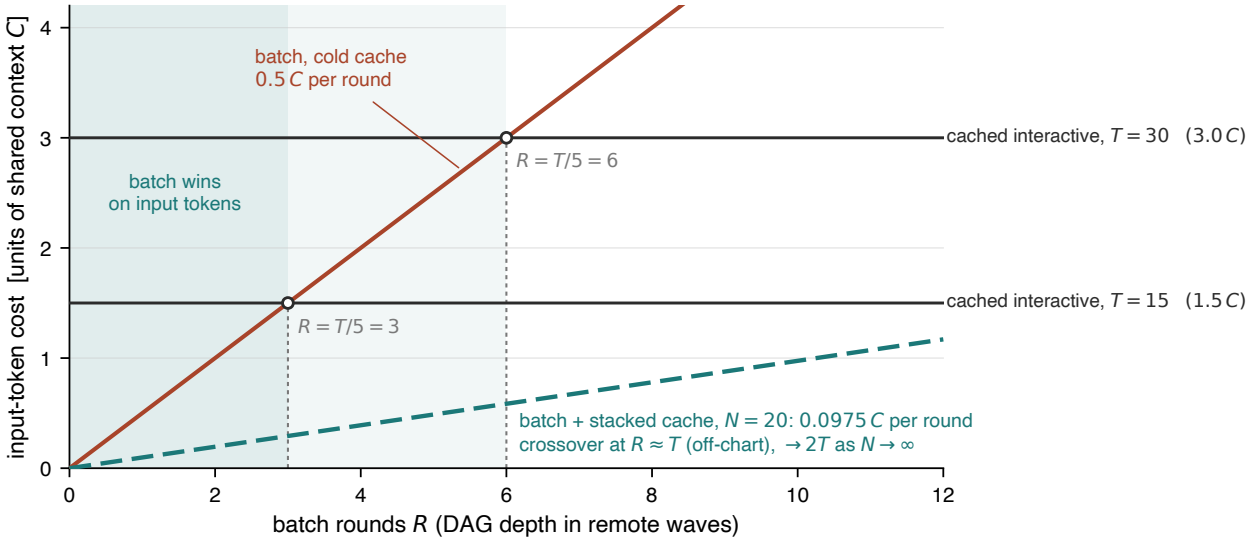


Figure 1: **The economics condition.** Input-token cost of completing one task, in units of its shared context C, as a function of batch rounds R. A prompt-cached interactive agent pays $\approx 0.1\,C$ per turn (horizontal lines for T=15 and T=30 turns). Uncached batch pays $0.5\,C$ per round: it beats the cached baseline only left of the R=$T/5$ crossovers. With batch and cache discounts stacked across a wave of N prefix-sharing items (dashed, N=20), the slope drops to $0.0975\,C$ and the crossover moves to $R \approx T$ (the $N \to \infty$ limit is $2T$): width substitutes for depth-collapse. Output tokens (not shown) favor batch unconditionally at ~50%.

3 The economics condition

Let a task require shared context C (repository slices, house rules, instructions). An interactive agent across T turns re-reads that context every turn; with prompt caching, the input bill is approximately $0.1\,C\,T$ (cache writes and the uncached first turn add small constants). This baseline is deliberately generous to the interactive side — it ignores cache-write premiums and the per-turn growth of the uncached conversation suffix, both of which raise the real interactive bill — so the condition below is conservative *against* batch. LazyCode front-loads the same context into each of R batch rounds at the batch rate: $0.5\,C\,R$, assuming cold caches across waves. Batch wins on input iff

$$0.5\,C\,R \;<\; 0.1\,C\,T \quad\Longleftrightarrow\quad \boxed{R < T/5}.$$

A 30-turn interactive task compiled to 2–3 batch rounds costs 1.0–1.5 C against the cached agent's 3.0 C; the same task compiled poorly, to 8 rounds, loses. Output tokens are never cacheable, so batch's 50% discount on them is unconditional — and agentic coding is output-heavy. The blended saving is therefore "~50% on output plus a conditional input win": 30–70% against a well-cached baseline, and the widely-quoted 60–85% only against an uncached one. The design's own kill condition follows: if realized `rounds_per_node` degrades toward interactive depth, the input economics invert.

3.1 Width changes the condition

The derivation above assumes batch calls never hit caches. Where batch and prompt-cache discounts *stack* — both providers' documentation implies it; we have not verified the line items — that assumption is wrong in the favorable direction, and Anthropic's 1-hour cache TTL exists precisely for workloads whose calls are separated by batch-scale queuing delays. Within one wave, N items sharing a hoisted prefix C pay approximately $C \cdot 0.5 \cdot 2$ once (a 1-hour cache write at the batch rate, Anthropic's pricing) and $C \cdot 0.5 \cdot 0.1$ for the remaining $N-1$ reads — *when caching is worth requesting at all*. Because the cache write costs 2×, a rational renderer caches only when the wave is wide enough to amortize it, giving a piecewise per-item cost:

$$\text{per-item input} \;\approx\; C\,R \cdot \min\!\left(0.5,\; 0.05 + \tfrac{0.95}{N}\right) \quad\Longrightarrow\quad \text{batch wins} \iff R \cdot \min\!\left(0.5,\; 0.05 + \tfrac{0.95}{N}\right) < 0.1\,T.$$

At N=1 the min selects the uncached branch and the condition is the original $R < T/5$; caching pays for $N \geq 3$; at N=20, $R < 1.03\,T$; as $N \to \infty$, $R < 2T$.

The arithmetic is Anthropic-specific (explicit cache-control with a 2× 1-hour write); OpenAI's automatic positional caching has no write premium, making its optimistic form closer to $R(0.05 + 0.45/N)$ if the discounts stack there — a provider-by-provider question the benchmark must settle empirically. Two further honesty notes: our own adapter does not yet request the 1-hour TTL (it sends the default 5-minute `ephemeral`; the upgrade is a one-line change, listed in Section 11), and whether the 50% batch discount applies to cache-read/write *line items* is implied by provider "stacking" language but unverified by us.

Two consequences. First, *width is the real currency*: a coding agent must manufacture it by splitting one task along independence boundaries, but fleet workloads (evaluation sweeps, corpus processing) have width by construction — the key fact behind Section 9's verdict. Second, cache hit rates are best-effort (providers cite 30–98%), and there is a concrete mechanism for missing the ideal: a cache entry becomes readable only after the request that writes it has begun returning, so wave items the provider schedules concurrently can *each* pay the write, degrading one-write-$(N-1)$-reads toward N writes. The honest statement is the sensitivity range, not a point estimate, and within-wave hit rate belongs in the benchmark suite as a first-class metric.

3.2 What ten dollars buys

The conditions above are stated in token multiples; a reader wants dollars. So fix a concrete workload and price it at published list rates.[1]

One unit of work W: an overnight test-coverage pass over 12 modules of one repository. Shared context C = 8,000 tokens (execution role, repository outline, house rules) is byte-identical across items; each

[1]Anthropic, *Pricing*: `https://platform.claude.com/docs/en/about-claude/pricing` — Claude Haiku 4.5 at $1.00/Mtok base input, $1.25 five-minute cache write, $0.10 cache read, $5.00 output; Message Batches at $0.50/$2.50. OpenAI, *Pricing*: `https://developers.openai.com/api/docs/pricing` — gpt-4o-mini at $0.15/Mtok input, $0.075 cached input, $0.60 output; Batch API at $0.075/$0.30. Both accessed 12 August 2026; both providers publish the batch tier as 50% off input *and* output. Prices move, so `bench/cost_report.py` carries the same sheet declaratively and the arithmetic can be re-run against a current one.

module contributes u = 4,000 tokens of unique context and receives o = 2,000 tokens of generated diff. The interactive agent works the modules sequentially over T = 36 turns (three per module) with prompt caching at a *generous* 100% hit rate — C written once and read 35 times, each module's u written once and read twice. LazyCode compiles the same unit into one wave of N = 12 items (R = 1 remote round per node) plus a free local VERIFY, and, conservatively, assumes *cold* batch caches: none of Section 3.1's stacking upside is booked. Both sides emit the same 24,000 output tokens — same model, same artifacts, same verifier — so the comparison holds quality fixed and varies only the tier.

Table 1: **One workload unit W, priced at published list rates.** Priced arithmetic over the cost model, *not* a measured invoice — no provider was billed for this table. The interactive column is prompt-cached at a generous 100% hit rate; the batch column assumes cold caches. "Input tokens billed" counts every token that meets a meter, which is why the interactive side pushes 3× the input volume for the same work: it re-reads C on every turn.

	Claude Haiku 4.5		gpt-4o-mini	
Per unit W	interactive	batch	interactive	batch
Input tokens billed	432 k	144 k	432 k	144 k
Output tokens billed	24 k	24 k	24 k	24 k
Input cost	\$0.1076	\$0.0720	\$0.0366	\$0.0108
Output cost	\$0.1200	\$0.0600	\$0.0144	\$0.0072
Cost per unit	**\$0.2276**	**\$0.1320**	**\$0.0510**	**\$0.0180**
Units bought by \$10	43.9	75.8	196	556
Saving	**42.0%**		**64.7%**	
\$10 of interactive work costs	**\$5.80**		**\$3.53**	

So: \$10 spent online-only buys 43.9 units of W on Haiku 4.5; the same \$10 spent through the batch tier buys 75.8, or equivalently the 43.9 units cost \$5.80 instead of \$10. Two things the table shows that the input-side model alone does not. First, *output dominates*: it is 53% of the interactive bill and 45% of the batch bill, and it is exactly where the 50% discount is unconditional — the contested input win rides on top of a saving that is mostly not contested at all. Second, the size of the win is a property of the *baseline's* cache discount rather than of batch: gpt-4o-mini's cached input is 50% of list, not the 0.1× Anthropic-style read the derivation in Section 3 assumes, so its prompt-cached baseline is weaker and batch looks better. Quoting only the larger number would reproduce the strawman of Section 2.

Everything above is priced arithmetic over the cost model, not a measurement. The instrument that does meter it ships — `bench/cost_report.py` runs one workload through both tiers and prices the metered ledger (Section 8) — and a real-provider spend measurement is still the named next step.

3.3 The flex-tier confrontation

OpenAI's flex tier delivers batch pricing on synchronous calls: one string, no architecture, minutes of extra latency instead of hours. Any defense of the batch-compilation architecture must therefore rest on what flex does *not* provide: the 24-hour window that potentially buys schedulable width (waves timed for cache stacking, subject to the hit-rate caveats of Section 3.1), the batch path's vendor-reported loop-depth and output-size advantages, cross-run pooling, and — on the self-hosted side — the co-serving economics of the companion system (Tidal), where the batch tier monetizes idle capacity a flex-style tier would still consume at peak. For pure per-call discount on an interactive loop, flex dominates; this paper's architecture earns its complexity only where those additional properties are the point.

4 Architecture

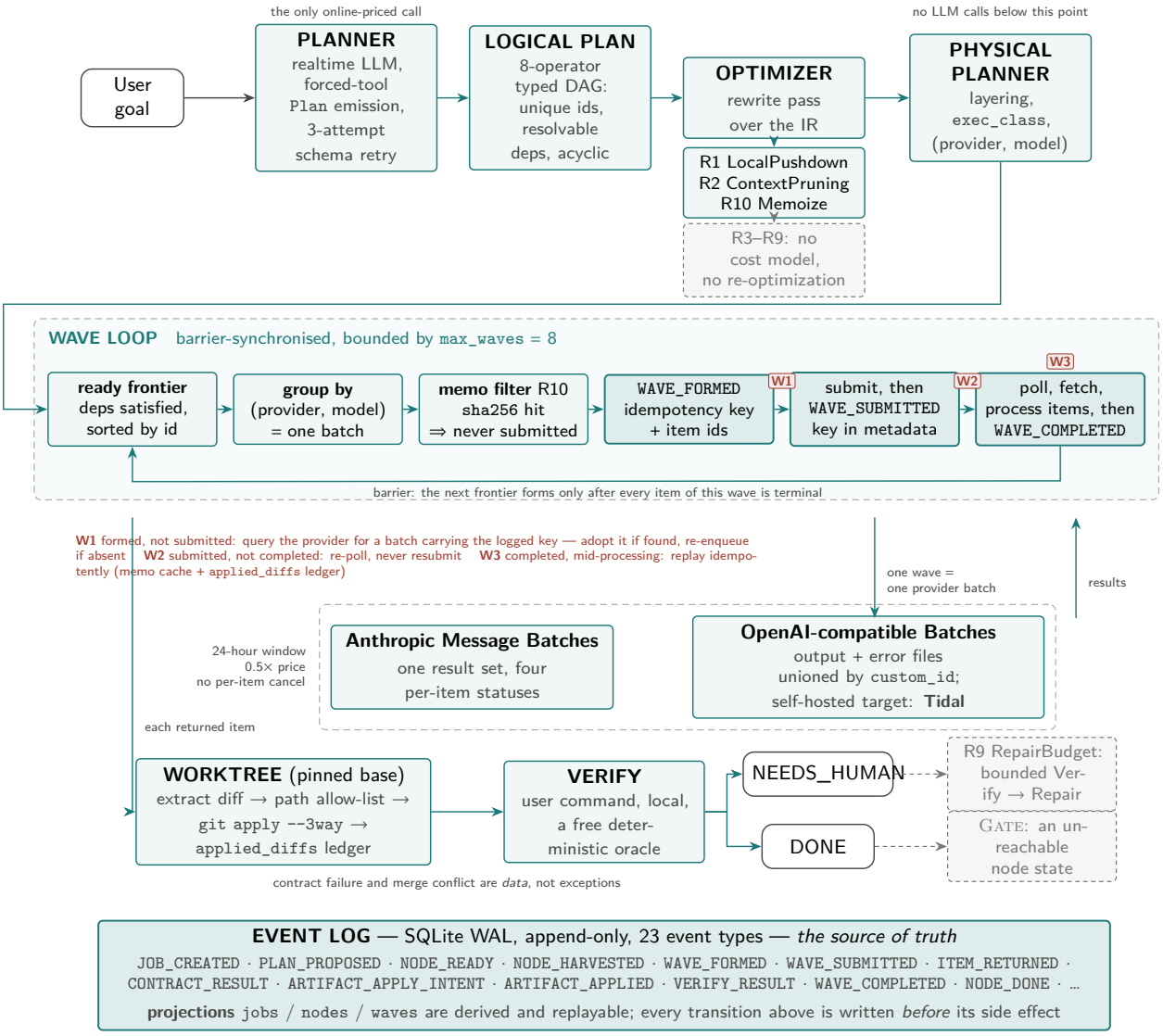


Figure 2: **The LazyCode pipeline.** Solid/accent components are shipped code; dimmed components are designed with plumbing but not yet live (Section 8). The event log is the source of truth; `jobs`/`nodes`/`waves` tables are replayable projections. W1–W3 mark the three crash-safety windows of Section 4.4.

4.1 The logical plan: an algebra of engineering work

Eight typed operators form the planning vocabulary: EXPLORE, DECOMPOSE, GENERATE, EDIT, VERIFY, JUDGE, REDUCE, GATE. Only GENERATE and EDIT carry a `context_spec` (what to harvest) and an `output_contract` (what shape must come back) — they are the operators that spend money and mutate the tree. The plan is a Pydantic-validated DAG: unique ids, resolvable dependencies (including `"node.*"` fan-out patterns), acyclicity by iterative DFS. Critically, the planner cannot drift from this schema, because the planning prompt's tool definition *is* `Plan.model_json_schema()` and the algebra documentation shown

to the model is generated from the model fields themselves; a validation error is fed back verbatim for up to three repair attempts.

The DAG-shape thesis is executable prompt text, quoted in full because it is the paper's mechanism in one sentence:

> *"Structure the work as a shallow, WIDE DAG (few dependency layers, much fan-out) because wall-clock is proportional to DAG depth: split independent work (per-file, per-module) into sibling nodes at the same layer instead of chaining it."*

4.2 The optimizer: rules with database analogues

Table 2 lists the ten rewrite rules with their database lineage. Two rewrites ship today, plus R10 — an execution-layer memoization rather than a plan rewrite; the table's Status column is load-bearing (Section 8).

Table 2: Optimizer rules, their database analogues, and implementation status. "Plumbed" = data model and call paths exist; the decision logic does not.

Rule	DB analogue	Status	Semantics
R1 LocalPushdown	predicate pushdown	**shipped***	planner-flagged EXPLORE executes locally, free, no LLM round (*the flag is trusted, not analyzed, and local execution today yields a scope artifact rather than tool output — Section 8)
R2 ContextPruning	projection pruning	**shipped**	nodes touching ≤2 literal files drop the repo map from their context
R3 FanoutWidening	join reordering	prompt-level	split coarse nodes along independence boundaries; today width comes from the planner prompt, not a rewrite
R4 PrefixFactoring	common subexpr. elim.	partial	shared context hoisted to a cache-hinted prefix block; per-call today, not per-wave
R5 ModelTiering	access-path selection	designed	cheapest model whose predicted verify-pass rate clears threshold; escalate on failure
R6 Vectorize	batched row processing	plumbed	pack k homogeneous tasks into one call; `call_items` maps 1 call to k nodes (k=1 today)
R7 Speculate	branch prediction	designed	submit enumerable continuations as siblings — restricted to *remote*-decider branches after review showed the local-decider case saves zero waves
R8 HedgeInsertion	hedged reads	designed	stall detection + realtime duplicate for critical-path items
R9 RepairBudget	—	designed	bounded Verify→Repair loops, then NEEDS_HUMAN
R10 Memoize	materialized views	**shipped**	`sha256(model, prompt, mode, sample_idx)` keys an exact-reuse cache; identical calls never re-bill

4.3 Wave formation and the barrier ruling

The scheduler runs a bounded loop over the *ready frontier*: nodes whose dependencies are all successful, sorted by id for deterministic apply order. Remote frontier nodes group by (provider, model); each group becomes exactly one provider batch — a *wave*. Waves are barriers: the whole frontier submits, the orchestrator polls to completion, results are processed, and only then does the next frontier form. The design review's most contested ruling chose barriers over rolling flushes, for two reasons that compound: the cost model stays literal (wall-clock $\approx$ depth $\times$ wave latency) and the acceptance test stays countable. Rolling execution is deferred, not rejected.

Wave identity is *content-addressed*: the idempotency key is a hash of the rendered items plus a flush ordinal recovered by scanning the durable log — not an in-memory counter — so a crashed-and-resumed orchestrator derives the same key and can adopt its own orphaned submission.

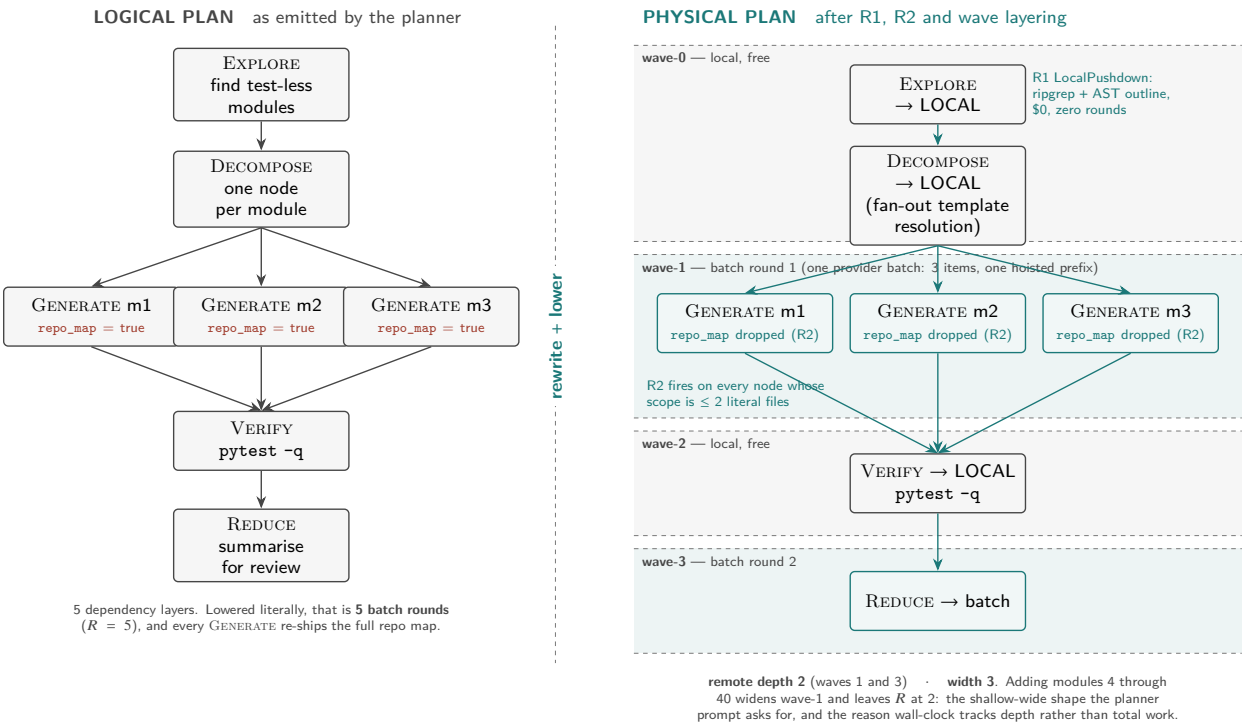


Figure 3: **A plan, compiled.** Left: the planner's logical DAG for "add tests for three modules." Right: after rewriting and lowering — the EXPLORE is pushed to free local execution (R1), the DECOMPOSE resolves locally as a fan-out template over EXPLORE's bindings (no LLM round; a judgment-requiring DECOMPOSE would add one), trivially-scoped GENERATE nodes shed their repo map (R2), and layering yields two remote rounds regardless of module count: depth stays constant while width scales.

4.4 Crash safety: three windows, three recoveries

A batch submission is a paid, remote, irrevocable side effect, and the system treats it with WAL discipline. `WAVE_FORMED` — carrying the idempotency key and item ids — is durably logged *before* any provider call; `WAVE_SUBMITTED` after; `WAVE_COMPLETED` only after every returned item is fully processed. Each window has a distinct recovery: (W1) formed-not-submitted resolves by querying the provider for a batch bearing the logged key — found means adopt, absent means re-enqueue; (W2) submitted-not-completed

re-polls and never resubmits; (W3) completed-mid-processing replays idempotently, because item results route through the memo cache and diff application checks a content-hash ledger before touching the worktree. Independent belts and braces: provider-side metadata, log-derived keys, the `applied_diffs` ledger, and a `UNIQUE(memo_key)` constraint on the spend cache. This is the strongest-implemented subsystem (Section 8), and deliberately so: in an architecture whose entire premise is unattended execution, recovery is the product.

5 From interactive intent to self-sufficient calls

Three mechanisms convert interactive intent into deferred execution.

Front-loaded harvest under byte budgets. Context is gathered *before* submission, deterministically: an AST-derived repository outline (demote-largest-then-drop under an 8 KB budget — breadth survives, detail degrades), the referenced files (60 KB each, 400 KB total, overflow explicitly listed as omitted), and house rules resolved from a fixed priority list of convention files. Determinism matters twice over: identical repo state yields byte-identical context, which is what makes the memo key (R10) an exact-reuse cache rather than a heuristic one.

Contracts on the way back. A GENERATE/EDIT node declares its output contract — today, a unified diff confined to a glob allow-list. Validation is defense-in-depth ordered: path normalization rejects absolute paths and traversal *before* glob matching (because `fnmatch`'s `*` happily matches `/` and `..`), then a three-way `git apply` against a pinned-base worktree, with rollback on conflict and a content-hash ledger making application idempotent. Contract failure and merge conflict both route to NEEDS_HUMAN — failure is data, not an exception.

The assumption ledger. The prompt contract states it plainly: *"You cannot ask questions — when you must make a judgment call, make the most reasonable choice and record it under an 'Assumptions:' heading."* The planner records its own scope-level judgment calls in the plan; executors append theirs after each artifact. What would be a blocking clarification in an interactive loop becomes a reviewable data artifact in the morning report. This is the interaction-model transplant in miniature — and, we will argue, the single most portable idea in the system (Section 9).

The tuning surface. The parameters that actually govern how intent decomposes today: the trivial-scope threshold for context pruning (2 files), the four harvest byte budgets, `max_waves` (the depth ceiling, default 8), the uniform (provider, model) assignment (which makes wave count equal DAG depth), sampling temperature 0 (which makes memoization sound), and the planner's schema-retry budget. A second tier — the cost slider λ, per-job budgets and deadlines, hedge timers, vectorize k — is parsed and stored but not yet consumed (Section 8); we list it as designed surface, not live control.

6 Provider reality

Three discoveries from building against real batch APIs, each of which reshaped the design.

(1) *No per-item cancellation exists* on either provider — only whole-batch cancel. The original hedging design assumed it; hedging was rebuilt as duplicate-and-discard: a realtime copy races the stalled batch item, first result wins at the node level, the loser is billed and dropped.

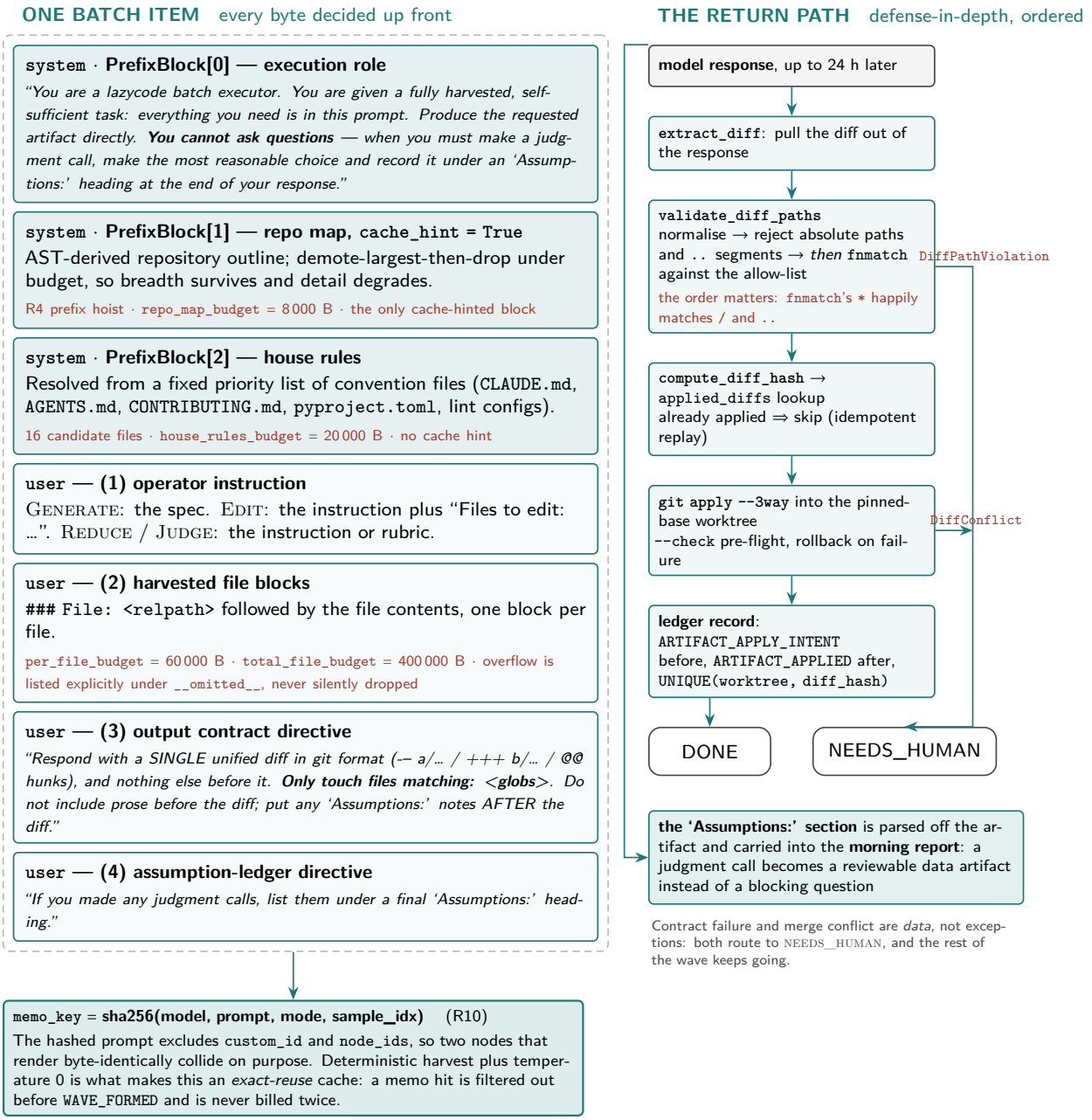


Figure 4: **Anatomy of a self-sufficient call.** Every byte the model will see is decided before submission, under explicit budgets; the output contract constrains the return shape; judgment calls come back as a structured *Assumptions* section feeding the morning report rather than blocking execution.

(2) *Idempotency must be provider-visible*: both adapters stamp the submission key into batch metadata — natively on the OpenAI wire (verified end-to-end against a live gateway), and via `extra_body` on Anthropic, where round-tripping of top-level batch metadata is *not documented* and our recovery test coverage is mock-level; until verified live, the Anthropic W1 recovery should be read as "re-enqueue," with adopt-orphan guaranteed only on OpenAI-compatible providers.

(3) *The two wire protocols differ where it hurts*: Anthropic streams one result set with four per-item statuses; OpenAI returns two files (output and error) that must be unioned by `custom_id`, reports no per-item expired status (it is inferred from batch state and count remainders), and silently omits cancelled items. The OpenAI-compatible adapter is what lets LazyCode execute against *self-hosted* batch tiers — including Tidal, the companion system that co-serves batch work on the same GPUs as online traffic, closing the loop from client intent to self-owned capacity.

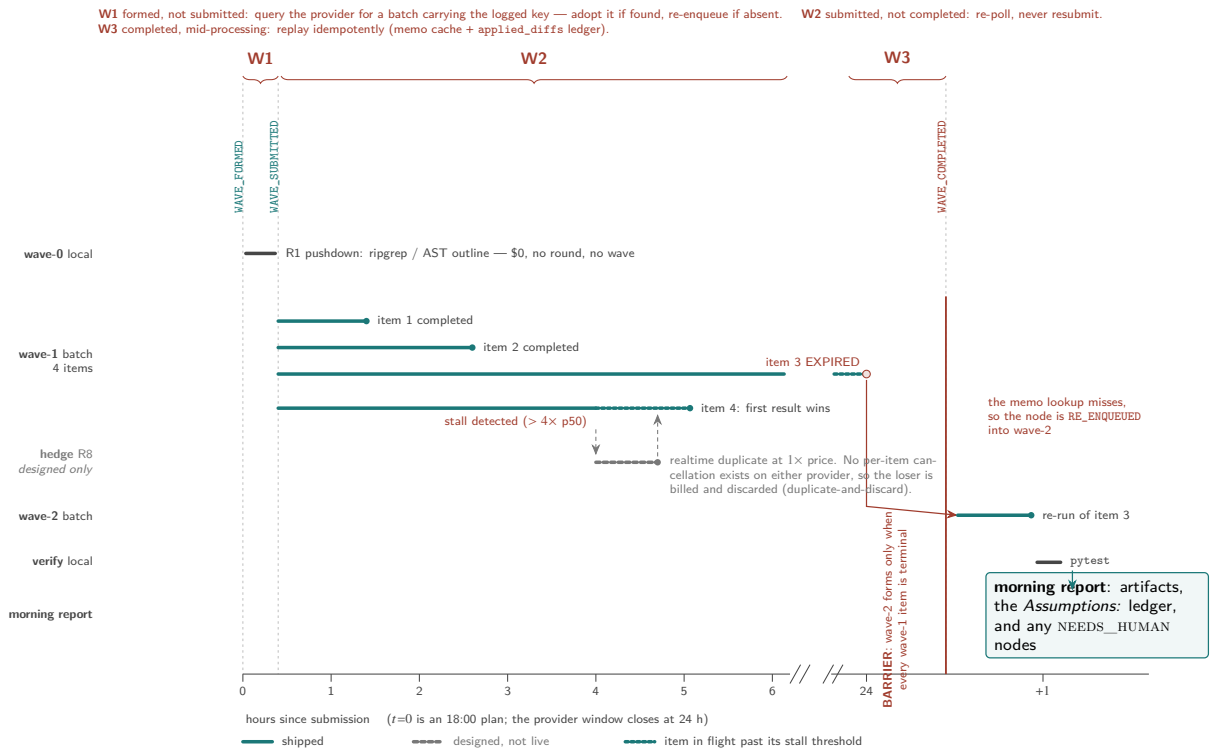


Figure 5: **Wave execution over a night.** Barriers separate waves; within a wave, items complete independently. The hedge path (dimmed: designed, not yet live) races a realtime duplicate against a stalled item under duplicate-and-discard semantics — the redesign forced by the absence of per-item cancellation. Expired items re-enter the next wave through the memo check. W1–W3 are the crash windows of Section 4.4.

7 Measured economics: receipts from a public batch lane

Between drafts of this paper, OpenRouter shipped a beta Batch API: one endpoint in front of the closed-model batch lanes, billing half the sync per-token price for the same model, with a 24-hour window and inline results. This gave the measurement campaign of Section 8 a metered public target months before the self-hosted one, and we took it. A fourth adapter (~300 lines plus 25 tests) speaks the wire; everything upstream — planner, optimizer, waves, store, verifier — is the code already described. All dollar figures

below are *receipts*: the provider-reported `usage.cost` of each batch and the per-generation billing records of each sync call, reconciled against the account's own usage meter — not list-price arithmetic.

Setup. Three arms per task, byte-identical prompts and one mechanical verifier across all three. *Interactive*: Claude Code headless, one session per task, routed through the same provider — the paper's honest baseline, an agent loop with tool use. *Compiled-sync*: the ablation control, not a product mode — LazyCode's own compiled single-shot calls executed at the full synchronous price, so that the compilation effect and the lane discount can be read separately. *LazyCode*: the same calls as one wave through the batch lane — the product. Models are three closed families with a clean 50% batch lane (Claude Haiku 4.5, GPT-5.4-mini, Gemini 3.7 Flash); the fan-out tier is HumanEval, a fixed 50-problem subset verified by its own assertion suites, with the agent free to iterate and the single-shot arms getting exactly one attempt. One seed, temperature 0; raw responses, batch ids, generation ids and ledgers are archived in the repository.

Table 3: Fan-out tier (HumanEval subset-50), receipted. "sync" and "batch" are the *same* compiled single-shot requests on the two price lanes (only "interactive" is a different execution model — the agent loop); "x/50" is the verifier pass count; $ is the provider-charged total for all 50 problems. The batch:sync cost ratio is what the invoice says, not what the price page promises.

model	arm	pass	receipted $	batch:sync
Haiku 4.5	interactive	49/50	3.3113	—
Haiku 4.5	sync	46/50	0.0721	—
Haiku 4.5	batch	47/50	0.0367	0.509
Gemini 3.7 Flash	sync	47/50	0.0552	—
Gemini 3.7 Flash	batch	49/50	0.0281	0.510
GPT-5.4-mini	sync	46/50	0.0520	—
GPT-5.4-mini	batch	47/50	0.0255	0.491

What the receipts say. Three results, in decreasing order of importance. First, *the compilation is the product*: the agent baseline was charged $0.066 per solved problem against $0.0014 for the same model given a self-sufficient single-shot prompt — a 46× gap that has nothing to do with batch pricing. The agent's three turns of tool traffic, file round-trips and conversation carry are what Section 5 compiles away. Second, *the 50% lane is real and stacks on top*: batch cost 50.9% of sync on Haiku, 51.0% on Gemini, and 49.1% on GPT-5.4-mini for identical request bodies, landing the blended interactive-to-batch ratio at 90× on this tier. Third, *quality holds where the paper predicted it would*: batch matched or beat sync on every family (the two lanes run the same weights), and the agent's iteration loop bought it +2 problems over the one-shot arms (49 vs. 46–47) — a real, priced advantage of ~$0.065 per extra solve here, and the honest counterweight the repair roadmap (Section 4.4) exists to close.

Two observations the paper could not have printed before. Batch turnaround is a *provider* property, not a lane property: every Anthropic- and Google-routed batch — 2 to 50 items — reached terminal state in 90 seconds to nine minutes against its 24-hour window, while the identically-shaped OpenAI-routed batch sat at 0 of 50 completed for its first 2.4 hours and finalized at 3.5 hours — a 25× spread between providers on one workload. The 24-hour window is the contract; the realized tail is who you are routed to — which is exactly why LazyCode prices the deadline, not the typical case (Section 3). And self-reported agent telemetry is not a receipt: the CLI reported $15.84 of list-price usage for sessions the provider actually billed at $3.31 — a 4.8× overstatement (cache discounts and routing below list), which is why every number in Table 3 comes from the biller.

The pipeline run, end to end. The fan-out tier exercises the economics; a separate run exercises the *claim*: that LazyCode converts an interactive-shaped goal into batch execution, unattended. The `add-type-hints` fixture task ran through the shipped pipeline against the live lane: plan proposed (planner's realtime call included), one wave formed and submitted, three items returned, three verify contracts passed, three diffs applied, repository-level verify green — JOB_DONE in 193 seconds, wave receipt $0.00309. The store's append-only event log is the evidence artifact, published with the run.

That run also put two design claims under live fire. The first attempt crashed mid-poll — the provider's just-created batch was not yet visible to reads and the adapter treated the 404 as fatal. Recovery behaved exactly as Section 4.4 claims: on resume, the W1 path adopted the already-paid batch by its idempotency key and completed the job with zero duplicate spend, the receipts showing a single batch. The lesson (post-create visibility lag is a provider reality) is now a bounded retry in the adapter. Second, every diff in that first wave passed its verify contract and then failed to *apply*: the model's unified diffs carried wrong hunk line-counts, which `git apply` rejects at parse time. The fix — try exact apply, fall back to `--recount`, keep the rollback semantics — took the wave from 0/3 to 3/3 applied and is the first accuracy improvement this system can attribute to a receipted production run rather than a code review.

Table 4: Online agent vs. LazyCode, head to head (Claude Haiku 4.5, receipted). "Quality" is the verifier: HumanEval pass count on the fan-out tier, official-harness resolved count on the deep tier. "Wall" is what one run costs in time: per-item mean for the sequential arms, whole-wave turnaround for batch (all items land together). The agent buys +2 solves on the fan-out tier and *loses* two on the deep tier; LazyCode pays 1–2% of the agent's bill on both. †Not a product mode: the ablation control — LazyCode's own compiled calls, byte-identical, executed at the full synchronous price so the compilation effect and the lane discount can be read separately.

tier	arm	quality	receipted $	$/solved	wall
fan-out	agent (Claude Code)	49/50	3.3113	0.0676	18 s/item
(HumanEval-50)	compiled, sync price†	46/50	0.0721	0.0016	2.2 s/item
	LazyCode (batch lane)	47/50	0.0367	0.0008	8.1 min/wave
deep	agent (Claude Code)	6/10	5.6923	0.9487	91 s/item
(SWE-bench ×10)	compiled, sync price†	8/10	0.2582	0.0323	39 s/item
	LazyCode (batch lane)	8/10	0.1291	0.0161	9.6 min/wave

The deep tier. The fan-out tier flatters the compilation (tiny prompts, no dependencies), so the study's second tier is the shape that should favor the agent: ten SWE-bench Verified instances from the benchmark's easiest difficulty band ("<15 min fix"), real repositories, resolution judged by the official docker harness. The single-shot arms get the *oracle* retrieval setting — the problem statement plus the one file the reference patch touches, emitting the complete corrected file (diffs are computed locally, so models are graded on the fix, not on diff syntax) — while the agent explores the checkout itself. The receipts: interactive resolved 6/10 for $5.69 (15.8 mean turns, $0.95 per resolved instance); sync resolved 8/10 for $0.258; batch resolved *the same* 8/10 for $0.129 — 0.500 of sync on the invoice, $0.016 per resolved instance, 59× below the agent's. Table 4 puts the two tiers side by side. Read fairly: oracle retrieval hands the one-shot arms the located file, and locating is part of what the agent's turns pay for; on deeper instances than this band the iteration should earn its cost back. But the direction is the finding — on well-scoped work, one compiled call with the right context beat fifteen turns of exploration, and the batch lane halved it again with an identical resolved set.

What this still does not show. Within-wave cache stacking (Section 3.1) remains unmeasured — fan-out prompts were too small to register it and the deep tier ran one call per instance — and `rounds_per_node`

stayed 1 by construction on both tiers. One seed per cell; SWE-bench beyond the easiest band, and the self-hosted lane, remain open. Section 11 keeps the full list.

8 Implementation truth

The system is 20,369 lines of Python (9,265 in the package, 7,987 in tests, 3,117 in the benchmark harness) with 440 default tests (442 including 2 deselected e2e tests). Three findings from auditing our own code, reported because papers rarely do and reviewers always should.

The optimizer is an architecture, not yet an optimizer. The shipped rewrite layer is 200 lines implementing two rules; R2's entire rewrite is one boolean flip under a hardcoded two-file threshold, and R1 trusts a planner-supplied flag rather than analyzing answerability — moreover its "free local execution" currently emits a question-and-scope artifact, not ripgrep/AST output (the operator schema has no `context_spec` to harvest), so fan-out driven by a local EXPLORE cannot yet resolve. There is no cost model, no adaptive re-optimization at wave boundaries, and `Caps`-aware splitting is validated post-hoc by adapters rather than planned around. The query-engine *shape* — typed IR, rewrite pass, physical lowering, execution — is real and tested; the query-engine *intelligence* is the roadmap. Relatedly, wave count equals DAG depth today only because a single model makes (provider, model) grouping degenerate, and the statically-computed layer assignment is discarded in favor of dynamically formed, content-addressed waves.

The state machine is ~40% aspirational. Of 21 declared node states, 8 are unreachable (hedging, speculation, repair, gating, cancellation); of 23 event types, 2 are never emitted. We keep the full vocabulary in the schema deliberately — events are append-only and renaming is a migration — but the paper's figures dim what the runtime cannot reach.

Crash safety is the strongest subsystem. Every claim in Section 4.4 traces to tested code paths — on the OpenAI-compatible path; the Anthropic W1 adopt-orphan recovery is mock-level (Section 6): the three windows each have a dedicated recovery with a dedicated test, including kill-9-mid-wave resume that adopts the orphaned provider batch rather than double-paying. Where the rest of the system is honest scaffolding, this part is load-bearing today — which is the right inversion for an unattended-execution product.

Evaluation status. A benchmark harness (fixture tasks, standard-benchmark tiers, a pricing model, and a Claude-Code-CLI baseline runner) ships, and Section 7 now reports a receipted real-provider run over a public batch lane. What remains derived rather than measured is narrower: the width economics' cache-stacking term, realized `rounds_per_node` on dependent shapes, and the self-hosted target now that the companion self-hosted batch tier provides a meterable execution target (Tidal's gateway recovers 69.3% of a node's steady-state offline ceiling at 1.18×/1.23× online p50/p99 on a rented A6000 in its own evaluation [11]): the same fixture tasks run end-to-end against self-hosted batch capacity with per-item token ledgers, alongside realized `rounds_per_node` and within-wave cache-hit measurements.

One piece of that campaign's instrument is built and running. `bench/cost_report.py` executes a single workload through *both* tiers — online-only, as a sequential realtime call per item, and lazycode's batch waves — from one plan through the same orchestrator, renderer and harvester, differing only in the adapter; meters the per-call token ledger split into shared prefix, per-item unique context and output; and prices both sides from a declarative sheet of the published list prices of Section 3.2. On metered mock workloads it reproduces this paper's predicted per-item input factor exactly on both branches of the $\min(\cdot)$: the default cold-cache run observes the flat 0.5 factor (the uncached branch, since batch never hits a cache by default), and with `--stack-cache` it observes 0.3667 against $\min(0.5,\ 0.05{+}0.95/3)$ at $N{=}3$ — and reports quality

equivalence as exact *by construction*, since both tiers replay the same fixture responses at temperature 0 (a plumbing check, not evidence about model quality). What the instrument cannot do is spend: the default run is a $0 dry run, its live mode is off by default and gated behind an explicit acknowledgment, and a real-provider spend measurement remains the named next step.

9 Should this generalize? Frameworks, and an honest verdict

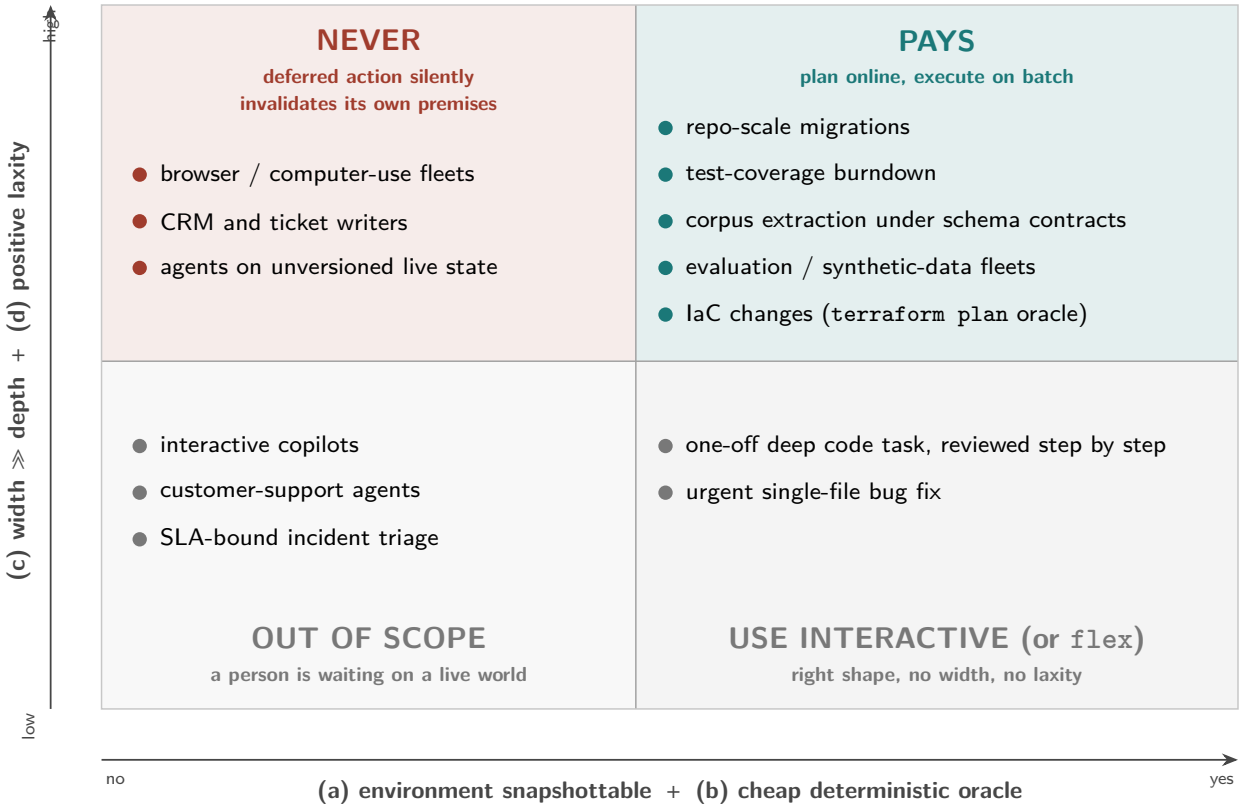
LazyCode is a coding CLI. The obvious question — asked of us, and by us — is whether plan-online/execute-on-batch belongs in general agentic frameworks: Pydantic AI with its durable-execution backends, NVIDIA's NeMo Agent Toolkit, LangGraph and kin. The answer we defend: *partially, and not as an architecture*.

The substrate exists; the seam does not. Durable-execution integration is a solved problem: Pydantic AI ships co-maintained DBOS, Temporal, Prefect and Restate backends behind a public capability interface, with cost-budget gating built in; DBOS alone provides exactly-once steps, checkpointed multi-day sleeps, deduplication ids (LazyCode's idempotency key, ready-made) and rate-limited queues — and its old Postgres objection has expired (SQLite is now the default).

What no framework has is a *first-class batch-API abstraction for model requests*: durable backends do wrap model calls in activities/steps (and Temporal can suspend turns on durable timers), but an activity-wrapped model call still completes as one synchronous unit — there is no protocol for terminating a run at a model call, pooling it into a provider batch, and resuming hours later from persisted history, the way Pydantic AI's deferred-*tool* protocol already does for tools. The single upstream change that would open this pattern to durable-execution frameworks is a `DeferredModelRequests` analogue of the existing deferred-tools protocol — terminate the run at a model call, resume from persisted history plus results. The transport machinery already ships; the protocol extension is small; an open issue has waited since May 2025 for exactly this design.

The negative control. NeMo Agent Toolkit — the most explicitly engineered framework in the field — has a genuinely strong profiler (its common-prefix detector independently rediscovers R4 from the serving side) yet no durable *mid-run* suspension, no cost-tier routing, and no provider-batch integration. The gap LazyCode fills is real across the ecosystem, not an artifact of coding CLIs. The only prior attempts at agent-side batch use are two 2026 fleet-pooling prototypes — both dispatcher-level, both stopping short of barriers, memoization, deadline machinery, and crash safety. Evaluation harnesses that *do* support batch APIs explicitly warn against using them for agentic tasks. We found no prior system that combines barriers, memoization, deadline machinery, and crash safety behind the batch tier.

The criterion. The architecture pays exactly where a workload satisfies four conditions: *(a) the environment is snapshottable* — LazyCode pins a git commit and applies diffs into an immutable fork, so an 18:00 plan is still valid at 03:00; a CRM-writing or browser agent acts on a live world that moves during the wave, and deferred action against an unversioned environment silently invalidates its own premises; *(b) a cheap deterministic oracle exists* — pytest and compilers make VERIFY free and local; where the only verifier is an LLM judge, failure is discovered a wave late and the repair round erases the discount; *(c) width dominates depth* — structurally within the task, or by inter-run concurrency (Section 3.1); *(d) laxity is positive* — nobody is waiting. Coding is the canonical member of this class — versioned by git, oracled by test suites, fan-out-able by file — but it is a member, not the definition. Repo-scale migrations, IaC changes (`terraform plan` as oracle), corpus extraction under schema contracts, and evaluation/synthetic-data fleets all qualify. Interactive copilots, SLA-bound triage, and live-environment actuation never will; naming that boundary is a contribution, because a naive generalization into those classes would do real damage.



(a) LazyCode pins a git commit and applies diffs into an immutable fork, so an 18:00 plan is still valid at 03:00.
(b) `pytest` and compilers make VERIFY free and local; where the only verifier is an LLM judge, failure surfaces a wave late and the repair round erases the discount.
(c) width comes structurally from the task or from inter-run concurrency (§3.1); once batch and cache discounts stack, width is the currency.
(d) nobody is waiting.

Figure 6: **Where plan-online/execute-on-batch pays.** The four-part criterion as a map. The architecture's home turf is the upper right; the upper left is the danger zone where deferred execution silently invalidates its own premises.

Transfer analysis and verdict. Component by component: the *wave executor* — barrier scheduling reinterpreted over N concurrent runs, memoization (R10 verbatim), prefix factoring (R4, now more valuable under discount stacking), vectorization plumbing (R6), laxity-driven {sync, flex, batch} routing with duplicate-and-discard hedging (R8), and the assumption ledger — transfers cleanly and is worth building as a ∼1–2k-line framework-agnostic library with thin DBOS/Temporal recipes and a Pydantic AI capability for cross-run pooling.

The *planner, operator algebra, and code-analysis rules* (R1–R3, R9, the harvester) do not transfer: their content is ripgrep, per-file independence, and free test oracles — irreducibly coding. Our recommendation, in ascending order of leverage: build the library; ship the pooling capability; and above all land the deferred-model-request protocol upstream, because it is small, it is the true blocker, and it would make the pattern available to durable-execution frameworks at once.

10 Related work

Semantic operators and LLM query optimization. LOTUS, Palimpzest, Abacus, DocETL and Aryn establish cost-based optimization over declarative LLM pipelines; they own the operator-DAG-with-optimizer idea for *data* processing. LazyCode's distinction is the workload class — stateful, side-effecting, tool-using engineering work against a versioned environment with contract verification — and the batch-API execution substrate. FrugalGPT and RouteLLM give the cost-tiering lineage for R5 (the verify-pass-rate-gated variant is ours but unimplemented); Halo and Helium consolidate multi-agent queries into optimized DAGs with KV-sharing on *local* serving — the same instincts, the opposite deployment; sleep-time compute is adjacent (idle-time precomputation, not deferred pricing).

Coding agents. SWE-agent's measured trajectory depths (≈13–15 turns) calibrate what T the economics condition must beat; OpenHands, Aider, and Agentless calibrate the interactive baseline space.

Batch tooling and fleet pooling. Curator, Ray Data LLM, and evaluation harnesses batch *independent* calls; none run an agentic loop through the batch tier, and the leading eval framework warns against trying. Two 2026 fleet-pooling prototypes mark independent convergence on inter-run batching, minus the systems machinery.

Database lineage. The transplant's ancestors are explicit — System R's cost-based optimization, Volcano/Cascades rewrite architecture, ARIES-style write-ahead recovery, and Saga-style compensation for long-lived transactions — and the paper's claim is the transplant, not the machinery.

The serving side, and the claim. The companion Tidal paper supplies the self-hosted batch tier — deadline-contracted co-serving on unmodified vLLM — that closes the loop this paper's adapter opens. The unoccupied conjunction LazyCode claims: a barrier-scheduled, memoized, crash-safe, deadline-aware batch executor driving a stateful, contract-verified agent against a snapshottable environment.

11 Limitations

The headline economics are now measured against a provider invoice on one public lane (Section 7), but at benchmark scale, one seed, on a fan-out-friendly tier: the dollar figures of Section 3.2 remain priced arithmetic, no self-hosted-lane run exists, and realized `rounds_per_node` is not instrumented (the current counter can only report 1), and within-wave cache hit rates — on which Section 3.1 leans — are best-effort

and unbooked. Provider latency tails are unpublished; the overnight guarantee is "most nights, with a priced fallback," not an SLA, and a hedge that fires re-buys realtime pricing for that item. The optimizer intelligence, hedging, repair, model tiering, vectorization $k>1$, and per-item cost accounting are designed but not live; the state machine carries their vocabulary ahead of their arrival. The flex tier undercuts the pure-discount motivation on one provider today. Three code-level debts surfaced by review: the adapter requests the default 5-minute cache TTL rather than the 1-hour TTL the width economics assume; Anthropic-side batch-metadata round-tripping (on which orphan adoption rests) is unverified against the live API; and local EXPLORE produces a scope stub rather than tool output. And the architecture's preconditions — git-snapshottable state, free oracles — bound its domain honestly: this is a compiler for a particular, valuable shape of work, not a universal agent runtime.

12 Conclusion

Data systems learned to go from batch to streaming; LazyCode runs the lesson in reverse for agents, and finds the pieces already lying around: a planning model that can emit typed DAGs, providers selling half-price deferred execution, database machinery for making it crash-safe, and — newly quantified here — discount stacking that rewards width more than heroic depth-collapse. The system's honest state is a strong skeleton with two shipped rewrite rules, its strongest-tested subsystem in recovery — a subsystem that has now recovered a real crashed job over a real batch lane without double-paying — and first receipts (Section 7) in place of previously unmeasured economics; its honest future is the rest of that campaign, including its companion self-hosted batch tier, and a small upstream protocol change that would let durable-execution agent frameworks defer a model call. The compiler metaphor sets the standard the project must ultimately meet: not that batch execution is architecturally workable — the shipped machinery and its crash-safety argue that much — but that the compilation is profitable, case by case, with the receipts to show it.

Acknowledgments

The design benefited from adversarial multi-model review; several of this paper's central corrections (the cached-baseline economics, the vacant-speculation example, the width-dependent refinement) were surfaced by reviews that set out to break the claims rather than confirm them.

References

[1] L. Patel et al. LOTUS: Semantic Operators for Bulk LLM Processing. VLDB 2025. arXiv:2407.11418.

[2] C. Liu, M. Russo, M. Cafarella et al. Palimpzest: Optimizing AI-Powered Analytics. CIDR 2025. arXiv:2405.14696.

[3] Abacus: Cost-Based Optimization for Semantic Operator Systems. arXiv:2505.14661.

[4] S. Shankar et al. DocETL: Agentic Query Rewriting for Complex Document Processing. arXiv:2410.12189.

[5] L. Chen, M. Zaharia, J. Zou. FrugalGPT. arXiv:2305.05176.

[6] I. Ong et al. RouteLLM: Learning to Route LLMs. arXiv:2406.18665.

[7] Halo: Holistic Agent-DAG Optimization for LLM Serving. arXiv:2509.02121.

[8] Helium: DAG-Consolidated Serving for Agentic Workflows. arXiv:2603.16104.

[9] K. Lin et al. Sleep-time Compute. arXiv:2504.13171.

[10] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. arXiv:2405.15793.

[11] G. Lunkupali Venugopal. Tidal: A Deadline-Contracted Batch Tier for Unmodified LLM Serving Engines. 2026. github.com/rajagurunath/tidal.

[12] Pydantic AI: Durable Execution (DBOS, Temporal, Prefect, Restate). pydantic.dev/docs/ai, 2026.

[13] DBOS: Durable Workflows. docs.dbos.dev, 2026.

[14] NVIDIA NeMo Agent Toolkit (v1.8). github.com/NVIDIA/NeMo-Agent-Toolkit, 2026.

[15] UK AISI. Inspect AI: Batch Mode. inspect.aisi.org.uk/models-batch.html.

[16] Anthropic. Message Batches and Prompt Caching documentation. platform.claude.com/docs, 2026.

[17] OpenAI. Batch API, Flex Processing, and Webhooks documentation. developers.openai.com, 2026.

[18] E. Sandler. Batch API is terrible for one agent. It might be great for a fleet. Apr 2026. github.com/erans/batching-harness.

[19] llmfleet: latency-budget routing to batched Anthropic calls. github.com/MukundaKatta/llmfleet, 2026.

[20] Bespoke Labs. Curator. github.com/bespokelabsai/curator.

[21] G. Graefe. Volcano — An Extensible and Parallel Query Evaluation System. IEEE TKDE, 1994.

[22] P. G. Selinger et al. Access Path Selection in a Relational DBMS. SIGMOD 1979.

[23] G. Graefe. The Cascades Framework for Query Optimization. IEEE Data Eng. Bull., 1995.

[24] C. Mohan et al. ARIES: A Transaction Recovery Method. ACM TODS, 1992.

[25] H. Garcia-Molina, K. Salem. Sagas. SIGMOD 1987.

[26] X. Wang et al. OpenHands: An Open Platform for AI Software Developers. arXiv:2407.16741.

[27] C. Xia et al. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489.

[28] Apache Spark: Adaptive Query Execution. spark.apache.org.